

the brain. Second, each of these matrices does serve a specific purpose in the following discussion.

Let us see how matrices can be represented and how some details about them can be obtained. Figure 3 shows how matrix A is represented with NumPy. Though they are not equivalent, we use the terms matrix and array as if they are synonyms.

I urge you to carefully learn how a matrix is created using the function `array` of NumPy. There is also a function `matrix` offered by NumPy to create two-dimensional arrays and matrices. However, this function will be deprecated in the future and its use is no longer encouraged. We have created matrix A and Figure 3 also shows some details about it. The line of code `A.size` tells us about the number of elements in the array. In this case, it is 9. The line of code `A.ndim` tells the dimension of the array. In this case, it is easy to see that matrix A is two-dimensional. The line of code `A.shape` gives us the order of matrix A. The order of a matrix is the number of rows and columns of that matrix. Notice that A is a 3 x 3 matrix (3 rows and 3 columns). Though I am not explaining any further, knowing the size, dimension, and shape of an array is a little tricky with NumPy. Figure 4 shows you why the size, dimension and shape of a matrix should be identified carefully. Minor changes while defining an array could lead to changes in its size, shape, and dimension. Hence, from Figure 4, it is clear that a programmer should be extra careful while defining matrices.

Now let us perform some basic matrix operations. Figure 5 shows how two matrices A and B are added. Indeed, there are two distinct Pythonic ways to *add* the two matrices, either using the function `add` of NumPy or the operator plus (+) for matrix addition. A very important point to remember is that two matrices can be added only if they are of the same order. For example, two 4 x 3 matrices can be added whereas a 3 x 4 matrix and a 2 x 3 matrix cannot be

```
>>> A=np.array([[1,2,3],[4,5,6],[7,8,9]])
>>> A
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
>>> A.size
9
>>> A.ndim
2
>>> A.shape
(3, 3)
```

Figure 3: Matrix A with NumPy

```
>>> X=np.array([[1,2,3],[4,5,6],[7,8]])
>>> X.ndim
1
>>> X=np.array([[1,2,3],[]])
>>> X.shape
(2,)
>>> X.size
2
>>> X=np.array([1,2,3])
>>> X.ndim
1
>>> X=np.array([[1,2,3]])
>>> X.ndim
2
>>> X=np.array([[[1,2,3]]])
>>> X.ndim
3
```

Figure 4: Size, dimension and shape of an array

```
>>> B=np.array([[1,2,3],[2,4,6],[3,6,9]])
>>> A.shape == B.shape
True
>>> A+B
array([[ 2,  4,  6],
       [ 6,  9, 12],
       [10, 14, 17]])
>>> np.add(A,B)
array([[ 2,  4,  6],
       [ 6,  9, 12],
       [10, 14, 17]])
>>> D=np.array([11,22,33])
>>> A+D
array([[12, 24, 36],
       [15, 27, 39],
       [18, 30, 41]])
```

Figure 5: Matrix addition

added. However, since programming is different from mathematics, NumPy does not follow this rule in practice. Figure 5 also shows how matrices A and D are added. Remember that, in the mathematical world, this sort of matrix addition is not valid. A rule called *broadcasting* decides how matrices of different orders should be added. We won't be discussing broadcasting at this point but if you are familiar with

C or C++, then typecasting of variables might give you a hint as to what happens when broadcasting rules are applied in Python. So, if you want to make sure that you are performing matrix addition in the true mathematical sense, it would be safe if the following test is performed and the answer is *True*:

`A.shape == B.shape`

It can be verified that if you try to add matrices D and H, then it will give you an error.

Please don't think that matrix addition is the only operation we are going to spend time on. Of course, there are a lot more matrix operations. Matrix subtraction and multiplication are illustrated in Figure 6. Notice that subtraction and multiplication also have two distinct Pythonic ways to perform the required operations. The function `subtract` or the operator for matrix subtraction (-) and the function `matmul` or the operator for matrix multiplication (@), can be used for performing matrix subtraction and multiplication, respectively. Execution of the operator * is also shown in Figure 3. Notice that the function `matmul` of NumPy and the operator @

perform matrix multiplication in the mathematical sense. So, be careful with the operator * while dealing with matrices.

Two matrices of order (m x n) and (p x q) can be multiplied if and only if n is equal to p. Further, the product matrix will be of order (m x q). Figure 7 shows more examples of matrices being multiplied. Notice that E@A is possible whereas A@E leads to an error. Carefully go through the examples of D@G and G@D. Using the shape attribute, identify